

Synaptic Depression Modeling

Data generation and model fitting

Jan Balewski, NERSC

May 12, 2026

Abstract

We describe a real-time adaptive Bernoulli state-space model (BSSM) for a network of interacting neurons with short-term synaptic depression (STD) and a lag- M synaptic memory kernel. The STD mechanism is represented as a latent dynamical resource variable that is updated at each discrete time step, coupled to observed binary spike indicators through a Bernoulli likelihood. For context, we also briefly review the Tsodyks–Markram (TM) resource model, which provides the continuous-time biological foundation for the discrete STD dynamics used in the BSSM.

1 Short-Term Synaptic Depression (STD) in Cortical Synapses

Short-term synaptic depression (STD) is a fast, reversible decrease in synaptic strength that occurs when a presynaptic cortical neuron fires repeatedly over short time windows (typically from milliseconds to seconds).

Activity-decaying phenomenon. When presynaptic firing is high, the postsynaptic response (EPSCs or IPSCs) becomes progressively smaller for successive spikes, producing a decaying envelope of synaptic efficacy during bursts. After presynaptic activity pauses, synaptic strength gradually recovers to baseline levels.

Main biological mechanism. STD is commonly modeled as depletion of synaptic “resources” required for effective neurotransmitter release. At each presynaptic spike, release depends on (i) the availability of readily releasable vesicles (or release-ready sites) and (ii) an effective probability of release (often related to intracellular calcium dynamics). Sustained activity depletes resources and reduces future release, while periods of silence allow recovery.

Relevant timescales. Recovery is typically on intermediate timescales (~ 100 ms to a few seconds), depending on synapse type and experimental protocol.

Functional consequences. STD acts as a rate-dependent synaptic filter: it attenuates sustained high-rate input, emphasizes transient changes in firing, and contributes to network stabilization by preventing runaway excitation.

2 Computational Models with STD

We describe two STD frameworks. The TM model (Section 2.1) is included for conceptual context only. The BSSM (Section 2.2) is the target implementation model.

The following notation is shared across both models: $U \in (0, 1]$ is the utilization parameter (fraction of available resources used per spike), and τ_{rec} is the STD recovery time constant.

Tsodyks–Markram short-term synaptic depression model

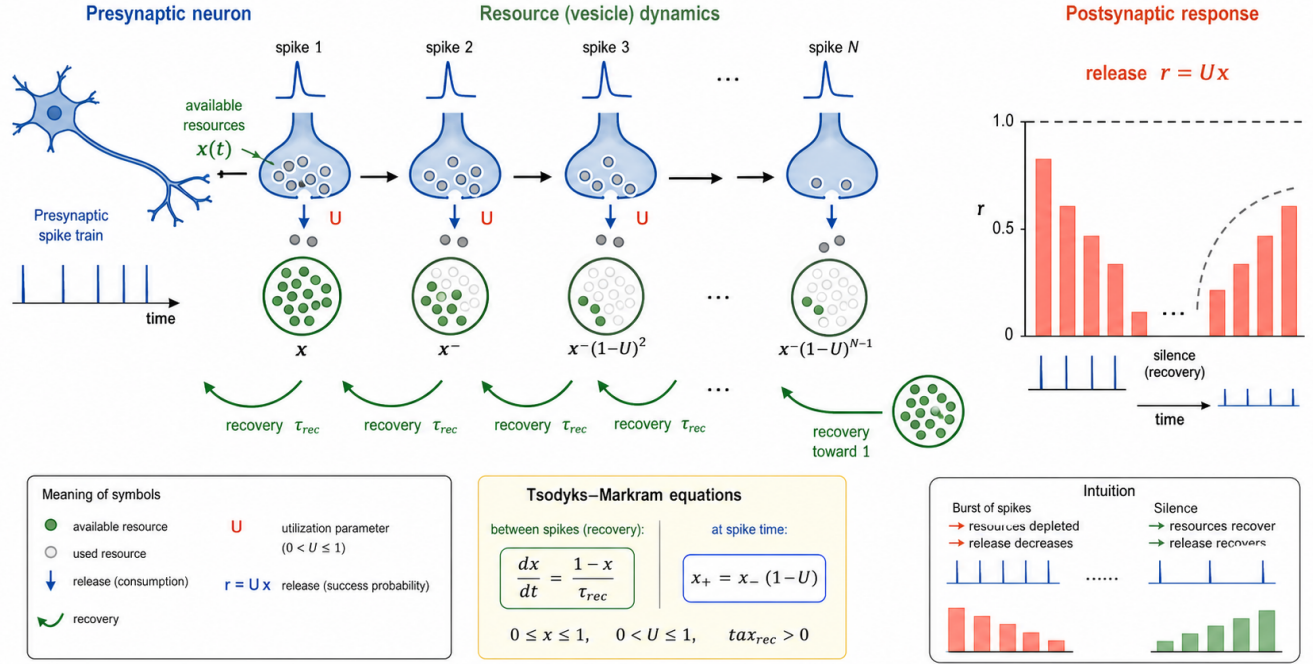


Figure 1: Cartoon of the Tsodyks–Markram STD resource model. Presynaptic spikes consume a fraction U of the available resource x , making successive release events smaller; during silence, the resource recovers toward $x = 1$ with time constant τ_{rec} .

2.1 Tsodyks–Markram (TM) Resource Model: Conceptual Background

The *Tsodyks–Markram (TM)* model [1] is a widely used continuous-time resource model for *short-term plasticity (STP)*. In the STD-only version, the synapse has a single *available-resource* variable $x(t) \in [0, 1]$. Between presynaptic spikes, resources recover exponentially:

The cartoon in Fig. 1 summarizes the STD-only TM mechanism: spikes multiply the available resource by $(1 - U)$, while inter-spike intervals allow the resource to recover toward one.

Between presynaptic spikes,

$$\frac{dx}{dt} = \frac{1-x}{\tau_{\text{rec}}}. \quad (1)$$

At a presynaptic spike time $t = t_k$, a fraction U of available resources is consumed:

$$x(t_k^+) = x(t_k^-) (1 - U), \quad (2)$$

and the released neurotransmitter amplitude at that spike is

$$r(t_k) = U x(t_k^-). \quad (3)$$

The TM model provides the biological motivation for the discrete STD resource update used in the BSSM below (Eq. 5), which is the exact discretization of Eq. (1) combined with Eq. (2).

Interpretation of U . The utilization parameter U is a fraction of the resources available at the moment of a spike, not a fixed fraction of the original full pool. For example, if $U = 0.5$ and several spikes arrive with negligible recovery between them, a fully recovered synapse releases 0.5 of the full pool on the first spike, then 0.25 on the second, then 0.125 on the third, and so on. Thus the axon can still generate more than two closely spaced spikes; STD only makes the effective synaptic release decrease geometrically during the burst.

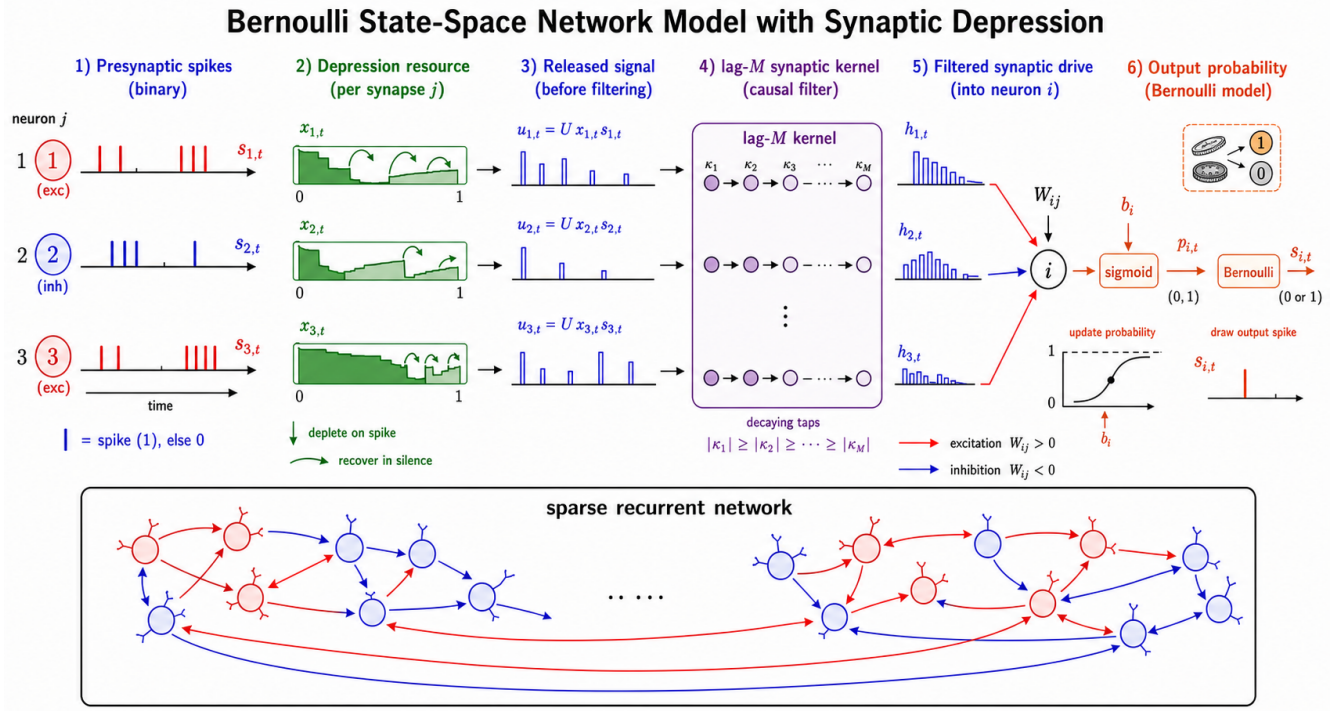


Figure 2: Cartoon of the full BSSM with STD and a lag- M synaptic kernel. Presynaptic binary spikes $s_{j,t}$ modulate the latent resource $x_{j,t}$, produce STD-weighted release $u_{j,t}$, pass through a causal synaptic kernel to form filtered drive $h_{j,t}$, and enter the postsynaptic Bernoulli observation model through signed recurrent weights W_{ij} and baseline B_i .

2.2 Real-Time Adaptive Bernoulli State-Space Network Model (BSSM) with STD and Lag- M Kernel

The *Bernoulli state-space model (BSSM)* represents a network of N neurons using (i) latent dynamical states encoding STD resources and (ii) a Bernoulli observation model for binary spike indicators conditioned on those states. This is the primary model of interest. At physiological firing rates (30–50 Hz) and bin width ($\Delta t = 1$ ms), the per-bin spike probability is $p \approx r\Delta t = 0.03$ – 0.05 . The corresponding Poisson probability of two or more spikes in one bin is approximately $p^2/2$, ranging from 4.5×10^{-4} to 1.25×10^{-3} . Thus the Bernoulli model is a good approximation for binary spike indicators while explicitly enforcing at most one spike per bin.

Figure 2 summarizes the sequential data-flow view of the model: the recurrent weighted input determines the postsynaptic spike probability, sampled presynaptic spikes generate release from the current STD resource, and that release updates both the next STD state and the filtered drive available in the next bin.

Discrete time and notation. Let $t \in \{0, 1, \dots, T-1\}$ index time bins of width Δt . For each neuron j , the primary observable is the binary *spike indicator*

$$s_{j,t} = \begin{cases} 1 & \text{neuron } j \text{ fired a spike in bin } t, \\ 0 & \text{neuron } j \text{ was silent in bin } t, \end{cases} \quad (4)$$

which records only *whether* a spike occurred. The downstream efficacy of that spike is determined separately by the latent STD resource variable $x_{j,t} \in [0, 1]$, which encodes the fraction of neurotransmitter available for release at the start of bin t (see Eq. 5). A spike that occurs when resources are depleted

($x_{j,t} \approx 0$) contributes little to postsynaptic drive, whereas a spike after full recovery ($x_{j,t} \approx 1$) contributes maximally.

STD latent state dynamics. Each presynaptic neuron j has its own latent resource variable $x_{j,t}$, tracking the STD state of that neuron’s outgoing synapses independently of all other neurons. The full network state is the collection $\{x_{j,t}\}_{j=1}^N$, with each variable evolving as the exact discretization of the TM continuous-time recovery (Eq. 1) combined with spike-triggered consumption (Eq. 2):

$$\begin{aligned} x_{j,t}^{\text{dep}} &= \underbrace{x_{j,t}(1 - U s_{j,t})}_{\text{spike-triggered depletion}} , \\ x_{j,t+1} &= \underbrace{1 - (1 - x_{j,t}^{\text{dep}})e^{-\Delta t/\tau_{\text{rec}}}}_{\text{exponential recovery after depletion}} , \quad j = 1, \dots, N. \end{aligned} \quad (5)$$

Release during bin t uses the pre-spike resource $x_{j,t}$. If a spike occurs, a fraction U is first consumed, and the depleted resource then recovers toward 1 over the interval to bin $t + 1$. Because the update depends only on neuron j ’s own spike indicator, resource variables of different neurons are conditionally independent given their respective spike histories, allowing all N updates to proceed in parallel.

STD-modulated release. The effective synaptic release from neuron j in bin t is:

$$u_{j,t} = U x_{j,t} s_{j,t}. \quad (6)$$

This quantity is zero in the absence of a spike, small when resources are depleted by recent activity, and large (up to U) after full recovery.

Lag- M synaptic kernel. Synaptic currents have temporal extent beyond a single bin. We convolve the STD-modulated release with a causal lag- M kernel $\boldsymbol{\kappa} = (\kappa_0, \dots, \kappa_{M-1})^\top$, $\kappa_\ell \geq 0$, encoding how a release event at lag $\ell + 1$ bins contributes to postsynaptic drive at time t . The effective STD-filtered synaptic input from neuron j at time t is:

$$h_{j,t} = \sum_{\ell=0}^{M-1} \kappa_\ell u_{j,t-1-\ell}. \quad (7)$$

Here $\ell = 0$ is one bin back and $M\Delta t$ is the finite lag-history span for the fast synaptic current. For an exponential synaptic kernel, practical choices take $M\Delta t$ to be several synaptic time constants τ_s so that the current tail is small at the truncation boundary. This lag window is distinct from the STD recovery memory: the slower recovery governed by τ_{rec} is carried by the latent state $x_{j,t}$ rather than by the finite synaptic kernel. Practical kernel choices include exponential decay, raised cosine basis functions, or a free learnable filter (see Section 3.2 for the generator choice).

Bernoulli observation model. For each postsynaptic neuron i , the conditional spiking probability is:

$$p_{i,t} = \sigma \left(B_i + \sum_{\substack{j=1 \\ j \neq i}}^N W_{ij} h_{j,t} \right), \quad (8)$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid, B_i is a baseline log-odds for neuron i , and W_{ij} is the synaptic weight from neuron j to neuron i (positive for excitatory, negative for inhibitory connections). Self-connections are excluded by the constraint $j \neq i$. The spike indicator is then drawn as:

$$s_{i,t} \sim \text{Bernoulli}(p_{i,t}). \quad (9)$$

For $t - 1 - \ell < 0$, pre-history values $u_{j,t-1-\ell}$ are set to zero, corresponding to a fully recovered synapse with no prior spiking. Because all kernel coefficients $\kappa_\ell \geq 0$ and all release values $u_{j,t} \geq 0$, the filtered drive satisfies $h_{j,t} \geq 0$ at all times; the implementation enforces this with a floor clamp after each recursive update.

Summary of model parameters. The full parameter set of the BSSM is $\theta = \{B_i, W_{ij}, \kappa_\ell, U, \tau_{\text{rec}}\}$, where B_i and W_{ij} are neuron- and synapse-specific, while this model assumes a single shared utilization parameter U for all presynaptic neurons and outgoing synapses. The recovery time τ_{rec} is also treated as shared in the generator and estimation procedure below. In the generator (Section 3), all parameters are fixed to known ground-truth values; in inference, they constitute the quantities to be estimated.

Real-time and adaptive aspects. The term *real-time* refers to the fact that both the latent STD state $x_{j,t}$ and parameter estimates can be updated sequentially as new spike indicators $s_{i,t}$ arrive, without reprocessing the full data history. The term *adaptive* refers to the property that $x_{j,t}$ automatically tracks recent presynaptic spiking through Eq. (5): resources deplete during bursts and recover during silence, so the effective synaptic gain $u_{j,t}$ adapts online to the presynaptic firing history. Together, these properties make the BSSM suitable for closed-loop neural decoding or real-time inference scenarios.

3 Data Generator: Cortical Network with STD

To validate the BSSM and test inference procedures, we generate synthetic spike trains from a network that mimics key properties of mammalian cortical dynamics. The generator uses the same BSSM structure as the inference model (Section 2.2), but with all parameters fixed to known ground-truth values, enabling direct comparison between true and estimated quantities.

3.1 Network Architecture

We consider a network of $N \in [50, 500]$ neurons partitioned following Dale’s law into $N_E = 0.8N$ excitatory and $N_I = 0.2N$ inhibitory neurons. The code constructs the initial synaptic matrix $W \in \mathbb{R}^{N \times N}$ with a spatial connections Dale generator. Neurons are placed on a two-dimensional grid over $[0, L] \times [0, H]$ with minimum spacing $d_{\min}$, and their order is sorted by position. For each presynaptic neuron j , an out-degree k_j is drawn from the configured range and k_j distinct postsynaptic targets are sampled without replacement with probability proportional to $D_{ij}^{-\delta}$, excluding $i = j$. This gives nearby neurons higher connection probability while keeping the diagonal zero.

The nonzero weights obey Dale’s law column-by-column:

$$W_{ij}^{(0)} = \begin{cases} +a_{ij} & \text{if neuron } j \text{ is excitatory,} \\ -\frac{N_E}{N_I} a_{ij} & \text{if neuron } j \text{ is inhibitory,} \end{cases} \quad a_{ij} \sim \text{Uniform}(1 - v, 1 + v), \quad (10)$$

for sampled edges only, with $W_{ii}^{(0)} = 0$. Finally, the whole matrix is rescaled to the requested spectral radius R ,

$$W = W^{(0)} \frac{R}{\rho(W^{(0)})}, \quad (11)$$

which preserves sparsity, signs, Dale’s law, and the zero diagonal.

3.2 Synaptic Kernel

We use an *exponential decay kernel* as the ground-truth synaptic filter:

$$\kappa_\ell = (1 - \alpha) \alpha^\ell, \quad \ell = 0, 1, \dots, M - 1, \quad (12)$$

where $\alpha = e^{-\Delta t/\tau_s}$ and τ_s is the synaptic time constant. The index $\ell = 0$ corresponds to the most recent lag (one bin back), and $\ell = M - 1$ to the oldest lag retained. This kernel corresponds to a standard single-compartment synaptic current model and is the exact discrete-time analog of the continuous-time shot-noise synaptic model. It is normalized so that $\sum_{\ell=0}^{M-1} \kappa_\ell \approx 1$ for $M\Delta t \gg \tau_s$.

A key computational advantage of this choice is that the lag- M convolution in Eq. (7) can be evaluated by a tail-corrected recursive update:

$$h_{j,t+1} = \alpha h_{j,t} + (1 - \alpha) u_{j,t} - (1 - \alpha) \alpha^M u_{j,t-M}, \quad (13)$$

with $u_{j,t-M} = 0$ when $t - M < 0$. This recurrence is exactly equivalent to the finite lag- M convolution for the exponential kernel and avoids explicitly summing over all M lags at every time step. The decay rate of κ_ℓ is set by the fast synaptic time constant τ_s , whereas the slower STD time constant τ_{rec} enters through the latent state update in Eq. (5). In the fitting code, this exponential kernel is fixed by command-line arguments: `--synaptic_tau` sets τ_s , and `--kernel_len_steps` sets the retained lag length M .

3.3 Ground-Truth Parameter Values

Table 1 lists the ground-truth parameter values used in the generator, motivated by mammalian cortical physiology.

Parameter	Symbol	Value	Rationale
Time bin width	Δt	1 ms	Resolves spikes; keeps p small
Synaptic time constant	τ_s	3–5 ms	AMPA-like fast excitation
STD recovery time	τ_{rec}	200–500 ms	Typical cortical STD
Utilization parameter	U	0.3–0.5	Shared moderate depression
Weight scale	v, R	$v = 0.4, R = 0.90$	Spectral-radius scaling
Baseline log-odds	B_i	sampled idle-rate range	User-adjusted baseline drive
Kernel length	M	$\gtrsim 3\text{--}5 \tau_s/\Delta t$	Fast synaptic lag window

Table 1: Ground-truth generator parameters. Ranges reflect variability across synapse types and network sizes. The kernel length M controls the finite lag history for the fast synaptic filter and is separate from the STD recovery time τ_{rec} . The baseline B_i is expressed as a log-odds consistent with the Bernoulli observation model (Eq. 8). Each neuron’s idle rate is drawn independently and uniformly from $[\text{idleRate}_{\text{min}}, \text{idleRate}_{\text{max}}]$, and the corresponding log-odds is $B_i = \log \frac{r_i \Delta t}{1 - r_i \Delta t}$.

3.4 Generation Procedure

Given the fixed parameters, synthetic spike indicators are generated sequentially for $t = 0, 1, \dots, T - 1$ as follows.

3.5 Network Stability Check

With recurrent excitatory connections, the network can exhibit runaway activity if synaptic weights are too large. Before generating long recordings, we recommend a short pilot run (~ 1 s) and verification that mean firing rates across neurons remain in the target range of 30–50 Hz. If rates exceed this range, one should reduce the spectral-radius target R or increase U : stronger STD provides a natural activity-dependent stabilizing mechanism by reducing synaptic efficacy during high-rate episodes.

4 Parameter Estimation: Two-Block Coordinate Descent

The true BSSM parameter set is $\theta = \{B_i, W_{ij}, \tau_{\text{rec}}, U, \tau_s\}$. The estimated set is $\tilde{\theta} = \{\tilde{B}_i, \tilde{W}_{ij}, \tilde{\tau}_{\text{rec}}, \tilde{U}\}$, with the exponential synaptic kernel fixed rather than fitted. In `fit5_BSSM_STD_blocks.py`, `--synaptic_tau`

Algorithm 1 Synthetic network spike train generator

```

1: Initialize:  $x_{j,0} \leftarrow 1$  for all  $j$  (fully recovered);  $h_{j,0} \leftarrow 0$  and  $u_{j,t} \leftarrow 0$  for  $t < 0$ ; draw  $W, \{B_i\}$  as in
   Eqs. (10)–(11).
2: for  $t = 0, 1, \dots, T - 1$  do
3:   for each neuron  $i = 1, \dots, N$  do
4:     Compute spiking probability:  $p_{i,t} \leftarrow \sigma(B_i + \sum_{j \neq i} W_{ij} h_{j,t})$  (Eq. 8)
5:     Draw spike indicator:  $s_{i,t} \sim \text{Bernoulli}(p_{i,t})$  (Eq. 9)
6:   end for
7:   for each neuron  $j = 1, \dots, N$  do
8:     Compute STD-modulated release from pre-spike resources:  $u_{j,t} \leftarrow U x_{j,t} s_{j,t}$  (Eq. 6)
9:     Update STD resource:  $x_{j,t+1} \leftarrow 1 - (1 - x_{j,t}(1 - U s_{j,t}))e^{-\Delta t/\tau_{\text{rec}}}$  (Eq. 5)
10:    Update recursive synaptic filter:  $h_{j,t+1} \leftarrow \alpha h_{j,t} + (1 - \alpha)u_{j,t} - (1 - \alpha)\alpha^M u_{j,t-M}$  (Eq. 13)
11:  end for
12: end for
13: Output: spike indicators  $\{s_{j,t}\}$ , ground-truth STD states  $\{x_{j,t}\}$ , weight matrix  $W$ .

```

sets τ_s and `--kernel.len.steps` sets M . The latent state $x_{j,t}$ is not a free parameter: given observed spikes and a candidate $(\tilde{U}, \tilde{\tau}_{\text{rec}})$, it is fully determined by the forward recursion (5). Consequently $\{x, u, h\}$ are recomputed from scratch whenever $\tilde{U}$ or $\tilde{\tau}_{\text{rec}}$ changes.

4.1 Initialization

Step 1 — $\tilde{\tau}_{\text{rec}}^{(0)}$ and $\tilde{U}^{(0)}$. Do not pre-estimate these from raw ISIs or spike-pair statistics. Set physiological starting values:

$$\tilde{\tau}_{\text{rec}}^{(0)} = 0.4 \text{ s}, \quad \tilde{U}^{(0)} = 0.4. \quad (14)$$

These initialize the forward STD recursion only; both are updated later in Block B.

Step 2 — **Burn-in.** Set $x_{j,0} = 1$ for all j and exclude the first

$$T_{\text{burn}} = \lceil \text{--burn_sec} / \Delta t \rceil. \quad (15)$$

bins from all likelihood evaluations (retain them for the forward state pass). The `--burn.sec` argument is mandatory and must be positive. This removes sensitivity to the initial condition, which decays on timescale τ_{rec} . The fitter treats the user-specified `-T` interval as the requested post-burn fitting duration and requires the input file to contain both the burn-in bins and the full requested fitting window. If the input is too short, the run aborts rather than shortening the requested fit.

Step 3 — $\{B_i^{(0)}, W_{ij}^{(0)}\}$. Initialize both $\{B_i^{(0)}, W_{ij}^{(0)}\}$ from the first $J = \text{--init.samples}$ post-burn bins. The `--init.samples` argument is mandatory and must be greater than 1000; the fitter aborts if fewer than J post-burn bins are available. The baseline log-odds use the mean firing rates in this initialization window:

$$B_i^{(0)} = \log \frac{\bar{p}_i}{1 - \bar{p}_i}, \quad \bar{p}_i = \frac{1}{J} \sum_{r=0}^{J-1} s_{i, T_{\text{burn}}+r}. \quad (16)$$

Weights are initialized from raw lag-1 spike covariances over the same post-burn window:

$$W_{ij}^{(0)} = \frac{1}{J-1} \sum_{r=1}^{J-1} (s_{i, T_{\text{burn}}+r} - \bar{p}_i^+) (s_{j, T_{\text{burn}}+r-1} - \bar{p}_j^-), \quad i \neq j, \quad (17)$$

with $W_{ii}^{(0)} = 0$. Here $\bar{p}_i^+$ and $\bar{p}_j^-$ are the means of the post- and pre-lag samples used in the covariance. Thus $W_{ij}^{(0)}$ is positive when spikes in neuron j one bin earlier are associated with increased spiking in

neuron i . The output NPZ stores these initialization arrays as `B_init` and `W_init`; it also stores the Eq. (17) means as `init_p_mean`, `init_p_plus`, and `init_p_minus`.

4.2 Block A: Update $\{B_i, W_{ij}\}$

Objective. With $(\tilde{U}, \tilde{\tau}_{\text{rec}})$ fixed and $\{h_{j,t}\}$ precomputed, the per-bin clipped logit and predicted probability are

$$\eta_{i,t} = \text{clip} \left(B_i + \sum_{j \neq i} \tilde{W}_{ij} h_{j,t}, -\eta_c, \eta_c \right), \quad \tilde{p}_{i,t} = \sigma(\eta_{i,t}), \quad (18)$$

where η_c is a logit-clip threshold (default $\eta_c = 10$). For reuse across the coordinate-descent blocks, define the mean Bernoulli negative log-likelihood over all post-burn bins and all neurons:

$$\mathcal{N}(\tilde{p}) = -\frac{1}{(T - T_{\text{burn}})N} \sum_{i=1}^N \sum_{t=T_{\text{burn}}}^{T-1} \left[s_{i,t} \log \tilde{p}_{i,t} + (1 - s_{i,t}) \log(1 - \tilde{p}_{i,t}) \right]. \quad (19)$$

Block A minimizes this NLL plus L1 regularization:

$$\mathcal{L}_A(\{B_i, \tilde{W}_{ij}\}) = \mathcal{N}(\tilde{p}) + \lambda_1 |\overline{\tilde{W}}|, \quad (20)$$

where $|\overline{\tilde{W}}|$ is the mean absolute value of the off-diagonal weights. The L1 term ($\lambda_1 > 0$, default 10^{-4}) promotes sparsity. The fitter saves the epoch-wise BCE term, L1 term, and their sum separately as `blockA_bce_loss`, `blockA_l1_loss`, and `blockA_loss`, respectively. The final fitted arrays are saved as `B_fit` and `W_fit`.

Optimizer. $\mathcal{L}_A$ is minimized by the Adam optimizer with learning rate η_W (default 10^{-2}), run for a fixed number of epochs. Each epoch draws a random permutation of the $T - T_{\text{burn}}$ post-burn bins and processes them in mini-batches of size B (default 8192).

Constraint applied after each mini-batch step.

1. **Zero diagonal:** $\tilde{W}_{ii} \leftarrow 0$ for all i (no self-connections).

Constraint applied after each full epoch.

2. **Spectral-radius cap:** if $\rho(\tilde{W}) > \rho_{\text{max}}$ (default 0.99), rescale $\tilde{W} \leftarrow \tilde{W} \cdot \rho_{\text{max}} / \rho(\tilde{W})$. The cap is delayed by `--delay_epoch_4_rhoMax`: if this value is D , the first D Block A epochs of each outer iteration skip the cap, and rescaling starts at epoch $D + 1$. This scalar rescaling preserves the zero diagonal already enforced after each mini-batch update.

4.3 Block B: Joint Update of $\tilde{U}$ and $\tilde{\tau}_{\text{rec}}$

With $\{B_i, \tilde{W}_{ij}\}$ fixed, the STD parameters are updated jointly by minimizing the mean Bernoulli NLL over a two-dimensional grid:

$$\mathcal{L}_B(\tilde{U}, \tilde{\tau}_{\text{rec}}) = \mathcal{N}(\tilde{p}(\tilde{U}, \tilde{\tau}_{\text{rec}})). \quad (21)$$

Here $\tilde{p}_{i,t}$ is evaluated by Eq. (18), and the drive $\{h_{j,t}\}$ is recomputed from scratch for each candidate pair $(\tilde{U}, \tilde{\tau}_{\text{rec}})$. No true values and no quadratic anchor terms are used.

Search procedure. A bounded two-dimensional grid-refinement search is used over $\tilde{U} \in [U_{\text{lo}}, U_{\text{hi}}]$ and $\tilde{\tau}_{\text{rec}} \in [\tau_{\text{lo}}, \tau_{\text{hi}}]$, supplied by the mandatory `--u_bounds` and `--tau_bounds` arguments. Each refinement round evaluates $\mathcal{L}_B$ on a Cartesian grid with `--u_grid_points` values in the U direction and `--tau_grid_points` values in the τ_{rec} direction (both default to 9). The best pair is retained, then the rectangular search box is narrowed by `--grid_shrink` (default 0.35) around that pair for `--grid_refine` refinement rounds (default 1). No gradients of $\mathcal{L}_B$ with respect to the STD parameters are used.

4.4 Outer Loop and Scheduling

The outer loop cycles through Blocks A→B for a user-specified number of iterations K_{outer} (default 6). To avoid early-stage bias from poor STD parameters corrupting the $\tilde{W}_{ij}$ fit, the first K_{freeze} outer iterations (default 2) run Block A only and skip Block B. The number of Block A epochs per outer iteration can differ between the frozen phase (E_{freeze} , default 40) and the live phase (E_{live} , default: same as E_{freeze}). The number of outer iterations and epochs are set by the user based on observed loss curves; no automatic stopping criterion is applied.

4.5 Computational Scaling

Each $\mathcal{O}(NT)$ forward pass (Eq. 5–13) is a sequential scan over T bins, vectorized across the N neurons. Block A processes $(T - T_{\text{burn}})$ time steps in mini-batches; the dominant cost is the $(T - T_{\text{burn}}) \times N$ matrix operations for η and the backward pass through Adam. Block B requires a forward pass for each pair in the joint $U \times \tau_{\text{rec}}$ grid. With current-best insertion, this is at most $(G_U + 1)(G_\tau + 1)(R + 1)$ forward passes per outer iteration; at the default $G_U = G_\tau = 9$ and $R = 1$, this is at most 200 forward passes.

4.6 Identifiability

The strongest coupling is between $\tilde{U}$ and $\tilde{W}_{ij}$: larger $\tilde{U}$ inflates the release amplitude $u_{j,t}$, which can be partially compensated by smaller weights. The bounded search domain $0 < \tilde{U} \leq 1$ prevents nonphysical values but does not remove this coupling by itself. A second coupling exists between $\tilde{\tau}_{\text{rec}}$ and $\tilde{W}_{ij}$: longer recovery produces more depleted $x_{j,t}$ trajectories, which Block A can partially offset by inflating $\tilde{W}_{ij}$. Neither coupling is an exact degeneracy because the model must match the temporal dynamics of recovery after bursts, not only the mean coupling level. The joint Block B search avoids an additional sequential-coordinate artifact between $\tilde{U}$ and $\tilde{\tau}_{\text{rec}}$, but it does not by itself eliminate the coupling between STD amplitude and recurrent weight scale. The frozen phase exploits this structure by stabilizing $\tilde{W}_{ij}$ before releasing the STD parameters.

References

- [1] M. V. Tsodyks and H. Markram. The neural code between neocortical pyramidal neurons depends on neurotransmitter release probability. *Proceedings of the National Academy of Sciences*, 94(2):719–723, 1997. doi:10.1073/pnas.94.2.719.